



CortexAI Master System Workflow Documentation

This document binds together the entire CortexAI ecosystem. It shows how the Frontend (React, Redux, Firebase), the Gateway (CORS, Proxy, Cookies), and the Microservices (Auth, Chat, Agent, Redis, MongoDB) are interconnected, illustrating how control and data switch from file to file.

1. Directory Tree & File Purpose Mapping

Below is the file structural map of the CortexAI workspace, detailing what each directory and file does:

```

cortexai/
├── extra/                                # Documentation Folder
│   ├── auth.readme.md                  # Auth Microservice Details
│   ├── creatingandapplyinglanggraph.readme.md # LangGraph Agent Details
│   ├── frontend.readme.md              # React SPA Client Details
│   ├── gateway.readme.md               # Gateway Proxy Details
│   ├── getcurrentuser.readme.md        # User session restore flow
│   ├── redis.readme.md                 # Redis caching setups
│   ├── redux.readme.md                 # React Redux global state config
│   ├── service.readme.md               # Microservices Layout Guide
│   ├── session,tokenmanagement.readme.md # Cookie/JWT lifecycle
│   └── workflowreadme.md               # [THIS FILE] Master System Guide
├── backend/                             # Backend Microservices Root
│   ├── package.json                    # Root dependencies config (ioredis)
│   ├── docker-compose.yml              # Starts local Redis container (Port 6379)
│   │
│   ├── gateway/                         # Gateway Proxy Service (Port 8000)
│   │   ├── index.js                    # Entry point, proxies endpoints
│   │   ├── controllers/
│   │   │   └── user.controller.js       # Handles local getcurrentuser routing
│   │   ├── middlewares/
│   │   │   └── auth.middleware.js       # protect guard checks cookies & Redis
│   │   ├── utils/
│   │   │   └── proxywithheader.js      # Decorator adds x-user-id header
│   │   └── package.json                 # Gateway dependencies
│   │
│   ├── shared/                          # Shared Backend Helpers
│   │   └── redis/
│   │       └── redis.js                 # Central ioredis client module
│   │
│   └── services/                         # Business Logic Servers
│       ├── auth/                         # Authentication Service (Port 8001)
│       │   ├── index.js                 # Auth server config
│       │   ├── config/
│       │   │   └── firebase.js          # Initializes Firebase Admin SDK
│       │   ├── controller/
│       │   │   └── auth.controller.js   # Handles login / logout operations
│       │   └── models/

```

```

|   |   |   └─ user.model.js           # Mongoose User Document Schema
|   |   |   └─ routes/
|   |   |       └─ auth.routes.js      # Route endpoints configuration
|   |   |   └─ serviceAccountKey.json  # Firebase Admin API private key credentials
|   |   |
|   |   └─ CHAT/                       # Chat History Service (Port 8002)
|   |       └─ index.js                # Chat server config
|   |       └─ config/
|   |           └─ db.js               # MongoDB connection helper
|   |       └─ controller/
|   |           └─ chat.controller.js   # Creates chats, saves messages
|   |       └─ model/
|   |           └─ conversation.model.js # Mongoose Conversation Schema
|   |           └─ message.model.js     # Mongoose Message Schema
|   |       └─ routes/
|   |           └─ chat.routes.js       # Route endpoints configuration
|   |
|   └─ agent/                          # LangGraph AI Service (Port 8003)
|       └─ index.js                   # Agent server config
|       └─ agents/
|           └─ chat.agent.js           # Groq chat agent logic node
|           └─ coding/search/pdf/ppt/etc. # Specialist sub-agent stubs
|       └─ config/
|           └─ db.js                  # MongoDB connection helper
|           └─ llmmodel.js            # Configures Groq/Gemini LLM drivers
|       └─ controller/
|           └─ agent.controller.js     # Controller executes graph and syncs history
|       └─ graph/
|           └─ graph.js                # StateGraph setup and compilation
|           └─ router.js               # LLM query categorizer routing node
|           └─ state.js                # LangGraph context schema definitions
|       └─ routes/
|           └─ route.js                # Post chat route configuration
|
└─ frontend/                          # Client Application Root
    └─ vite-project/                  # Vite React SPA (Port 5173)
        └─ utils/
            └─ axios.js                # Custom Axios client (withCredentials)
            └─ firebase.js             # Client Firebase config (popups)
        └─ src/

```

```
| | └─ main.jsx # Provider wrapper injects Redux store
| | └─ App.jsx # Fires getcurrentuser check on startup
| | └─ components/
| | | └─ sidebar.jsx # Lists chats, profile panel, logout
| | | └─ chatarea.jsx # Renders message bubble transcripts (stub)
| | | └─ arficate.jsx # Renders artifact panel outputs (stub)
| | └─ features/
| | | └─ getcurrentuser.js # API call to Gateway /me
| | | └─ createconversation.js # API call to create conversation
| | | └─ logout.js # API call to Gateway logout
| | └─ redux/
| | | └─ store.js # Aggregates Redux state slices
| | | └─ userslice.js # Manages user profile state
| | | └─ conversationslice.js # Manages chat and conversation selection
└─ package.json # Frontend libraries dependencies
```

2. Infrastructure Setup Matrix

The microservices are segmented across ports and run separate environments:

Service	Port	Primary Database	Key Dependencies / Drivers	Purpose
Gateway	8000	Redis (port 6379)	express , cors , cookie-parser , express-http-proxy	Gateway proxy, cookie validation, headers injection
Auth Service	8001	MongoDB auth	firebase-admin , mongoose , ioredis	User management, secure cookie distribution, Redis storage
Chat Service	8002	MongoDB chat	mongoose	Conversation indexing and chat transcript

Service	Port	Primary Database	Key Dependencies / Drivers	Purpose
				storage
Agent Service	8003	MongoDB agent	@langchain/langgraph , @langchain/google-genai , axios	Multi-agent coordination, LLM routing, history sync
Frontend	5173	Local Redux Store	react-redux , @reduxjs/toolkit , firebase , axios	Client UI dashboard, Google Sign-in modal, state binding

3. Docker-Compose Cache Specification

The Redis caching container is spun up inside the backend environment using Docker. Below is the blueprint of the active `docker-compose.yml` config file:

```
services:
  redis:
    image: redis          # Pulls the official Redis container image from Docker Hub
    ports:
      - 6379:6379        # Binds internal container port 6379 to local host port 6379
```

4. Environment Configurations (.env Blueprints)

For the microservices to communicate, they require specific environment keys. Below are templates for each server layer:

A. Gateway .env (backend/gateway/.env)

```
PORT=8000
FRONTEND_URL=http://localhost:5173      # Allow CORS requests from frontend
AUTH_SERVICE=http://localhost:8001      # Location of the Auth microservice
CHAT_SERVICE=http://localhost:8002      # Location of the Chat microservice
AGENT_SERVICE=http://localhost:8003     # Location of the Agent microservice
REDIS_URL=redis://localhost:6379        # Cache lookup connection string
```

B. Auth Service .env (backend/services/auth/.env)

```
PORT=8001
MONGODB_URL=mongodb+srv://.../auth      # MongoDB database for user accounts
FRONTEND_URL=http://localhost:5173      # CORS configuration origin
REDIS_URL=redis://localhost:6379        # Redis connection to write sessions
```

C. Chat Service .env (backend/services/CHAT/.env)

```
PORT=8002
MONGODB_URL=mongodb+srv://.../chat      # MongoDB database for conversation structures
FRONTEND_URL=http://localhost:5173      # CORS configuration origin
REDIS_URL=redis://localhost:6379        # Redis lookup URL
```

D. Agent Service .env (backend/services/agent/.env)

```
PORT=8003
MONGODB_URL=mongodb+srv://.../agent     # MongoDB database for agent session records
FRONTEND_URL=http://localhost:5173      # CORS configuration origin
REDIS_URL=redis://localhost:6379        # Redis lookup URL
GROQ_API_KEY=gsk...                     # Secret Groq model authorization key
GOOGLE_API_KEY=AQ...                     # Secret Google Generative AI key
CHAT_SERVICE=http://localhost:8002      # Location of the Chat microservice
```

E. Frontend Client .env (frontend/vite-project/.env)

```
VITE_SERVER_URL=http://localhost:8000      # Gateway endpoint URL
VITE_FIREBASE_API_KEY=AIZA...              # Public client Firebase key
```

5. Detailed API Endpoint Routing Mapping

The table below maps every backend HTTP endpoint across the system:

Route Path	HTTP Method	Handled By	Request Params / Payload
/auth/login	POST	Auth Service	{ token } (JWT in Body)
/auth/logout	POST	Auth Service	N/A (Requires cookie)
/me	GET	Gateway	N/A (Requires cookie)
/chat/create-conversation	GET	Chat Service	N/A (Requires header x-user-id)

Route Path	HTTP Method	Handled By	Request Params / Payload
/chat/get-conversation	GET	Chat Service	N/A (Requires header x-user-id)
/chat/update-conversation	POST	Chat Service	{ id, title }
/chat/save-message	POST	Chat Service	{ conversationId, role, content }
/chat/get-message/:conversationId	GET	Chat Service	:conversationId path parameter
/agent/chat	POST	Agent Service	{ prompt, conversationId }

6. Frontend State Tree & Mongoose Model Schema Structures

A. Redux State Tree JSON Blueprint

```
{
  "user": {
    "userData": {
      "_id": "603f721e72d2be2b78d2b271",
      "firebaseUid": "google-oauth-uid-12345",
      "name": "Jane Doe",
      "email": "janedoe@gmail.com",
      "avatar": "https://lh3.googleusercontent.com/avatar-url"
    }
  },
  "conversation": {
    "conversations": [
      {
        "_id": "603f739072d2be2b78d2b279",
        "title": "React vs Next.js",
        "userId": "603f721e72d2be2b78d2b271",
        "createdAt": "2026-07-15T04:20:00Z"
      }
    ],
    "selectedConversation": {
      "_id": "603f739072d2be2b78d2b279",
      "title": "React vs Next.js",
      "userId": "603f721e72d2be2b78d2b271"
    }
  }
}
```

B. Mongoose Document Blueprints

- **User Schema** (`models/user.model.js` inside Auth):
 - `firebaseUid` (String, unique): Stores Google authentication identifier.

- `name` (String)
 - `email` (String)
 - `avatar` (String)
 - **Conversation Schema** (`model/conversation.model.js` inside Chat):
 - `title` (String, default: "New Chat"): Renders in sidebar.
 - `userId` (String): References MongoDB user ID.
 - **Message Schema** (`model/message.model.js` inside Chat):
 - `conversationId` (ObjectId referencing Conversation): Groups message to a chat container.
 - `role` (String, enum: `["user", "assistant"]`): Determines message bubble styling.
 - `content` (String): Raw text.
-

7. Security & Browser Cookie Rules (Preventing XSS & CSRF)

The Gateway and Auth Services coordinate to secure browser cookie data exchanges:

- **`httpOnly: true` (XSS Protection):**
 - *Mechanism:* Restricts the `session` cookie from being read or manipulated by client-side Javascript scripts (like `document.cookie`).
 - *Benefit:* Blocks malicious browser extensions or cross-site scripting scripts from stealing the Session ID UUID.
- **`sameSite: "strict"` (CSRF Protection):**
 - *Mechanism:* The browser will only send this cookie on requests originating from the exact same site.
 - *Benefit:* Stops cross-site request forgery attacks where third-party links try to trigger backend APIs on behalf of a logged-in user.
- **`withCredentials: true` (Credentials Context):**
 - *Mechanism:* Enabled in the frontend custom Axios client and Gateway CORS settings.
 - *Benefit:* Instructs the browser to automatically package the cookies in AJAX requests, enabling the Gateway's validation middleware to extract and query them.

8. Firebase Google JWT Payload Structure

This JSON represents the decrypted structure of the token payload returned by the Google provider and verified by the Firebase Admin SDK inside `auth.controller.js` :

```
{
  "iss": "https://securetoken.google.com/cortexai-project",
  "aud": "cortexai-project",
  "auth_time": 1782000000,
  "sub": "firebase-google-sub-key-12345",
  "email": "janedoe@gmail.com",
  "email_verified": true,
  "name": "Jane Doe",
  "picture": "https://lh3.googleusercontent.com/avatar-url",
  "uid": "google-oauth-uid-12345"
}
```

9. LangGraph State Variable Transitions

As control flows from node to node inside the AI Graph, the `agentState` object (defined inside `state.js`) changes as follows:

1. **State on Invocation** (Initiated by `agent.controller.js`):

```
{ "prompt": "Write a quicksort in JavaScript", "conversationId": "convo-101" }
```

2. **State after Router Node** (Executed by `router.js`):

```
{ "prompt": "Write a quicksort in JavaScript", "conversationId": "convo-101", "agent": ... }
```

3. **State after Specialist Node** (Executed by `coding.agent.js`):

```
{
  "prompt": "Write a quicksort in JavaScript",
  "conversationId": "convo-101",
  "agent": "coding",
  "aiResponse": "Here is the quicksort implementation: \n```js\n..."
}
```

4. **State at End Node** (Returned back to `agent.controller.js`):

```
{
  "prompt": "Write a quicksort in JavaScript",
  "conversationId": "convo-101",
  "agent": "coding",
  "aiResponse": "Here is the quicksort implementation: \n```js\n..."
}
```

10. LangGraph State Wiring & Edge Trajectories

The `graph.js` file handles compiling and building the relationships between nodes:

- **Node Registration:**

- Nodes are mapped to their corresponding JavaScript functions:

```
workflow.addNode("router", router);
workflow.addNode("chat", chat);
workflow.addNode("search", searchagent);
```

- **The Conditional Edge:**

- Tied to the `router` node using a callback function.
- Reads `state.agent` string and routes execution to the corresponding node:

```
workflow.addConditionalEdges("router", (state) => state.agent, {  
  chat: "chat",  
  search: "search",  
  coding: "coding",  
  ...  
});
```

- **Node Fall-Through Edge:**

- The `search` node terminates by flowing directly into the `chat` node:

```
workflow.addEdge("search", "chat");
```

- **Termination Edge:**

- Nodes like `chat` and `coding` link directly to the ending state `_end_`, which returns the payload to the calling process.

11. End-to-End Master Workflows (File-to-File Control Switches)

Workflow 1: The Login & Onboarding Flow

When a user signs in for the first time.

1. [Trigger] home.jsx
 - └ User clicks login. googlelogin() fires Firebase popup and gets Google JWT ID token.
2. [API Client] home.jsx → axios.js
 - └ handlelogin(token) forwards token to Gateway `/auth/login``.
3. [Router] gateway/index.js
 - └ Gateway proxies request to Auth Service (Port 8001) based on `/auth`` URL prefix.
4. [Controller] auth.routes.js → auth.controller.js
 - └ Verifies token signature via Firebase Admin config inside firebase.js.
 - └ MongoDB query inside user.model.js: Checks if user exists. Creates record if new.
 - └ Generates secure session ID UUID: "sessionid".
 - └ Writes user details JSON to Redis via shared/redis/redis.js.
 - └ Sets HttpOnly cookie named "session" containing the UUID.
 - └ Responds with user profile details (status 200).
5. [State Sync] home.jsx → redux/userslice.js
 - └ Receives status 200 response.
 - └ Dispatches: `dispatch(setUserdata(data))` to update global state.
 - └ home.jsx re-renders: closes login modal, mounts SideBar and ChatArea.

Workflow 2: The Auto-Authorization Flow

When a user reloads or returns to the page.

1. [Trigger] App.jsx
 - └ Component mounts. useEffect calls getuser() helper.
 2. [API Client] getcurrentuser.js
 - └ Sends GET request to Gateway `/me``. Browser automatically attaches the secure session cookie.
 3. [Middleware] gateway/index.js → gateway/middlewares/auth.middleware.js
 - └ Gateway intercepts request. protect middleware reads the session cookie.
 - └ Queries Redis: `redis.get("session-" + sessionId)`.
 - └ If cached user data is returned, parses and binds it to req.user context.
 4. [Controller] gateway/controllers/user.controller.js
 - └ getCurrentuser controller executes, reading req.user and responding with the profile.
 5. [State Sync] App.jsx → redux/userslice.js
 - └ Receives profile. Dispatches: `dispatch(setUserdata(data))`.
 - └ Redux store variable userData updates. sidebar.jsx re-renders showing user profile.
-

Workflow 3: Chat Message & AI Agent Flow

When a user sends an AI prompt.

1. [Trigger] chatarea.jsx

└ User submits text prompt. Posts { prompt, conversationId } payload to Gateway `/api/chat`.
2. [Gateway Routing] gateway/index.js

└ Gateway protect middleware runs (checks cookie in Redis, populates req.user).

└ Calls proxyWithHeader() to inject "x-user-id" header holding req.user.userid.

└ Proxies request downstream to Agent Service (Port 8003).
3. [Save Prompt] agent.controller.js (Agent Service)

└ Receives request. Sends axios POST request to ``${CHAT_SERVICE}/save-message``.
4. [Write History] chat.controller.js (Chat Service)

└ Receives message body, creates Message document in MongoDB referencing Conversation.
5. [LangGraph Loop] agent.controller.js → graph/graph.js

└ Agent Service invokes LangGraph workflow: `graph.invoke({ prompt, conversationId })`.

└ Node `graph/router.js` prompts Groq model inside `config/llmmodel.js` to categorize intent.

└ Router returns selected agent (e.g. "chat"). Writes to state: `state.agent = "chat"`.

└ Node `agents/chat.agent.js` prompts Groq LLM to generate the final response.

└ Writes answer to state: `state.aiResponse = response.content`.

└ Graph ends, returning completed state object to agent.controller.js.
6. [AI Output Handback] agent.controller.js → Gateway → chatarea.jsx

└ Controller responds with { aiResponse } to Gateway, which routes it back to frontend.

└ React updates conversation Redux slice, rendering the user and AI message bubbles.

12. How Data Variables Transform Across Files

To ensure data integrity, variables switch names and storage locations as they move through the microservices system:

Phase	Variable Name	Type / Format	File Location	Purpose
Frontend	token	String	home.jsx	Encrypted

Phase	Variable Name	Type / Format	File Location	Purpose
Auth		(Google JWT)		passport verifying identity on login
Backend Auth	<code>decoded</code>	JSON Object	<code>auth.controller.js</code>	Decrypted Google details containing Firebase UID
Session Cache	<code>sessionid</code>	String (UUID)	<code>auth.controller.js</code>	Index key used to store/retrieve details in Redis
Session Key	<code>session-sessionid</code>	Redis String Key	<code>redis.js</code>	Cached string containing user ID, name, email, avatar
Browser Cookie	<code>session</code>	HTTP Header Cookie	<code>auth.middleware.js</code>	Client-side pointer containing session ID UUID
Context User ID	<code>req.user.userid</code>	Object Property	<code>auth.middleware.js</code>	Authenticated database User ID bound to request

Phase	Variable Name	Type / Format	File Location	Purpose
				scope
Inter-Service	x-user-id	HTTP Header	proxywithheader.js	Decorator header that passes User ID to microservices
Database Sync	userId	Schema Field	chat.controller.js	Connects conversations & messages to MongoDB profile

13. Developer Commands (Starting the Stack)

Follow these CLI steps to launch the local development environment:

1. Start Redis Database:

- Navigate to the root backend folder and launch the Redis container in background detached mode:

```
cd backend && docker-compose up -d
```

2. Start Backend Gateway (Port 8000):

```
cd backend/gateway && npm run dev
```

3. Start Auth Microservice (Port 8001):

```
cd backend/services/auth && npm run dev
```

4. Start CHAT Microservice (Port 8002):

```
cd backend/services/CHAT && npm run dev
```

5. **Start Agent Microservice (Port 8003):**

```
cd backend/services/agent && npm run dev
```

6. **Start React Frontend (Port 5173):**

```
cd frontend/vite-project && npm run dev
```

14. Fault Tolerance & Exception Handling (When Things Fail)

To maintain stability across services, specific error paths are defined:

- **Redis Down / Connection Loss:**

- *Where:* Gateway `auth.middleware.js`.
- *Behavior:* If the Redis container fails or drops offline, the `protect` middleware catches the connection error and returns a `500` response:
`{ message: "protect error", error }`. The user is safely prevented from accessing microservices with unchecked sessions.

- **Session Expiration / Missing Cookie:**

- *Where:* Gateway `auth.middleware.js`.
- *Behavior:* If `sessionId` cookie is missing or has expired in Redis, the middleware returns `401 Unauthorized` (or `session expired`). The frontend React app catches this status and redirects the user to the Google Login panel.

- **LLM API Failures / Timeout:**

- *Where:* Agent Service `agent.controller.js`.
- *Behavior:* If Groq or Gemini API endpoints time out or fail, the controller catches the error in its try-catch block, logging it locally and returning a `500` status payload to the client so that the frontend loading spinners terminate.